

Design an ATM Let's design an ATM. **We'll cover the following:**

- System Requirements
- How ATM works?
- Use Case Diagram
- Class Diagram
- Activity Diagrams
- Sequence Diagram
- Code

An automated teller machine (ATM) is an electronic telecommunications instrument that provides the clients of a financial institution with access to financial transactions in a public space without the need for a cashier or bank teller. ATMs are necessary as not all the bank branches are open every day of the week, and some customers may not be in a position to visit a bank each time they want to withdraw or deposit money.

ATM

System Requirements

The main components of the ATM that will affect interactions between the ATM and its users are:

1. **Card reader:** to read the users' ATM cards.
2. **Keypad:** to enter information into the ATM e.g. PIN. cards.
3. **Screen:** to display messages to the users.
4. **Cash dispenser:** for dispensing cash.
5. **Deposit slot:** For users to deposit cash or checks.
6. **Printer:** for printing receipts.
7. **Communication/Network Infrastructure:** it is assumed that the ATM has a communication infrastructure to communicate with the bank upon any transaction or activity.

The user can have two types of accounts: 1) Checking, and 2) Savings, and should be able to perform the following five transactions on the ATM:

1. **Balance inquiry:** To see the amount of funds in each account.
2. **Deposit cash:** To deposit cash.
3. **Deposit check:** To deposit checks.
4. **Withdraw cash:** To withdraw money from their checking account.
5. **Transfer funds:** To transfer funds to another account.

How ATM works?

The ATM will be managed by an operator, who operates the ATM and refills it with cash and receipts. The ATM will serve one customer at a time and should not shut down while serving. To begin a transaction in the ATM, the user should insert their ATM card, which will contain their account information. Then, the user should enter their Personal Identification Number (PIN) for authentication. The ATM will send the user's information to the bank for authentication; without authentication, the user cannot perform any transaction/service.

The user's ATM card will be kept in the ATM until the user ends a session. For example, the user can end a session at any time by pressing the cancel button, and the ATM Card will be ejected. The ATM will maintain an internal log of transactions that contains information about hardware failures; this log will be used by the ATM operator to resolve any issues.

1. Identify the system user through their PIN.
2. In the case of depositing checks, the amount of the check will not be added instantly to the user account; it is subject to manual verification and bank approval.
3. It is assumed that the bank manager will have access to the ATM's system information stored in the bank database.
4. It is assumed that user deposits will not be added to their account immediately because it will be subject to verification by the bank.
5. It is assumed the ATM card is the main player when it comes to security; users will authenticate themselves with their debit card and security pin.

Use Case Diagram

Here are the actors of the ATM system and their use cases:

Operator: The operator will be responsible for the following operations:

1. Turning the ATM ON/OFF using the designated Key-Switch.
2. Refilling the ATM with cash.
3. Refilling the ATM's printer with receipts.
4. Refilling the ATM's printer with INK.
5. Take out deposited cash and checks.

Customer: The ATM customer can perform the following operations:

1. Balance inquiry: the user can view his/her account balance.
2. Cash withdrawal: the user can withdraw a certain amount of cash.
3. Deposit funds: the user can deposit cash or checks.
4. Transfer funds: the user can transfer funds to other accounts.

Bank Manager: The Bank Manager can perform the following operations:

1. Generate a report to check total deposits.
2. Generate a report to check total withdrawals.
3. Print total deposits/withdrawal reports.
4. Checks the remaining cash in the ATM.

Here is the use case diagram of our ATM system:

Use Case Diagram for ATM

Class Diagram

Here are the main classes of the ATM System:

- **ATM:** The main part of the system for which this software has been designed. It has attributes like 'atmID' to distinguish it from other available ATMs, and 'location' which defines the physical address of the ATM.
- **CardReader:** To encapsulate the ATM's card reader used for user authentication.
- **CashDispenser:** To encapsulate the ATM component which will dispense cash.
- **Keypad:** The user will use the ATM's keypad to enter their PIN or amounts.
- **Screen:** Users will be shown all messages on the screen and they will select different transactions by touching the screen.
- **Printer:** To print receipts.
- **DepositSlot:** User can deposit checks or cash through the deposit slot.
- **Bank:** To encapsulate the bank which owns the ATM. The bank will hold all the account information and the ATM will communicate with the bank to perform customer transactions.
- **Account:** We'll have two types of accounts in the system: 1)Checking and 2)Saving.
- **Customer:** This class will encapsulate the ATM's customer. It will have the customer's basic information like name, email, etc.
- **Card:** Encapsulating the ATM card that the customer will use to authenticate themselves. Each customer can have one card.
- **Transaction:** Encapsulating all transactions that the customer can perform on the ATM, like BalanceInquiry, Deposit, Withdraw, etc.

Class Diagram for ATM UML for ATM

Activity Diagrams

Customer authentication: Following is the activity diagram for a customer authenticating themselves to perform an ATM transaction:

Activity Diagram for ATM Customer Authentication **Cash withdraw:** Following is the activity diagram for a user withdrawing cash:

Activity Diagram for ATM Cash Withdraw **Deposit check:** Following is the activity diagram for the customer depositing a check:

Activity Diagram for ATM Deposit Check **Fund transfer:** Following is the activity diagram for a user transferring funds to another account:

Activity Diagram for ATM Fund Transfer

Sequence Diagram

Here is the sequence diagram for balance inquiry transaction:

Sequence Diagram for ATM

Code

Here is the skeleton code for the classes defined above:

Enums and Constants: Here are the required enums, data types, and constants:

```
from enum import Enum
```

```
class TransactionType(Enum):  
    BALANCE_INQUIRY, DEPOSIT_CASH, DEPOSIT_CHECK, WITHDRAW, TRANSFER = 1, 2, 3,  
    4, 5
```

```
class TransactionStatus(Enum):  
    SUCCESS, FAILURE, BLOCKED, FULL, PARTIAL, NONE = 1, 2, 3, 4, 5, 6
```

```
class CustomerStatus(Enum):  
    ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, CLOSED, UNKNOWN = 1, 2, 3, 4,  
    5, 6, 7
```

```
class Address:  
    def __init__(self, street, city, state, zip_code, country):  
        self.__street_address = street  
        self.__city = city  
        self.__state = state  
        self.__zip_code = zip_code  
        self.__country = country
```

Customer, Card, and Account: “Customer” encapsulates the ATM user, “Card” the ATM card, and “Account” can be of two types: checking and savings:

```
# For simplicity, we are not defining getter and setter functions. The reader can  
# assume that all class attributes are private and accessed through their  
# respective  
# public getter methods and modified only through their public methods function.
```

```
class Customer:  
    def __init__(self, name, address, email, phone, status):  
        self.__name = name  
        self.__address = address  
        self.__email = email  
        self.__phone = phone  
        self.__status = status  
        self.__card = Card()  
        self.__account = Account()  
  
    def make_transaction(self, transaction):  
        None  
  
    def get_billing_address(self):  
        None
```

```

class Card:
    def __init__(self, number, customer_name, expiry, pin):
        self.__card_number = number
        self.__customer_name = customer_name
        self.__card_expiry = expiry
        self.__pin = pin

    def get_billing_address(self):
        None

```

```

class Account:
    def __init__(self, account_number):
        self.__account_number = account_number
        self.__total_balance = 0.0
        self.__available_balance = 0.0

    def get_available_balance(self):
        return self.__available_balance

```

```

class SavingAccount(Account):
    def __init__(self, withdraw_limit):
        self.__withdraw_limit = withdraw_limit

```

```

class CheckingAccount(Account):
    def __init__(self, debit_card_number):
        self.__debit_card_number = debit_card_number

```

Bank, ATM, CashDispenser, Keypad, Screen, Printer and DepositSlot: The ATM will have different components like keypad, screen, etc.

```

from abc import ABC

```

```

class Bank:
    def __init__(self, name, bank_code):
        self.__name = name
        self.__bank_code = bank_code

    def get_bank_code(self):
        return self.__bank_code

    def add_atm(self, atm):
        None

```

```

class ATM:

```

```

def __init__(self, id, location):
    self.__atm_id = id
    self.__location = location

    self.__cash_dispenser = CashDispenser()
    self.__keypad = Keypad()
    self.__screen = Screen()
    self.__printer = Printer()
    self.__check_deposit = CheckDeposit()
    self.__cash_deposit = CashDeposit

def authenticate_user(self):
    None

def make_transaction(self, customer, transaction):
    None

class CashDispenser:
    def __init__(self):
        self.__total_five_dollar_bills = 0
        self.__total_twenty_dollar_bills = 0

    def dispense_cash(self, amount):
        None

    def can_dispense_cash(self):
        None

class Keypad:
    def get_input(self):
        None

class Screen:
    def show_message(self, message):
        None

    def get_input(self):
        None

class Printer:
    def print_receipt(self, transaction):
        None

class CheckDeposit:

```

```

    def __init__(self):
        None

class CashDeposit:
    def __init__(self):
        None

class DepositSlot(ABC):
    def __init__(self):
        self.__total_amount = 0.0

    def get_total_amount(self):
        return self.__total_amount

class CheckDepositSlot(DepositSlot):
    def get_check_amount(self):
        None

class CashDepositSlot(DepositSlot):
    def receive_dollar_bill(self):
        None

```

Transaction and its subclasses: Customers can perform different transactions on the ATM, these classes encapsulate them:

```

from abc import ABC

class Transaction(ABC):
    def __init__(self, id, creation_date, status):
        self.__transaction_id = id
        self.__creation_time = creation_date
        self.__status = status

    def make_transation(self):
        None

class BalanceInquiry(Transaction):
    def __init__(self, account_id):
        self.__account_id = account_id

    def get_account_id(self):
        return self.__account_id

```

```

class Deposit(Transaction):
    def __init__(self, amount):
        self.__amount = amount

    def get_amount(self):
        return self.__amount

class CheckDeposit(Deposit):
    def __init__(self, check_number, bank_code):
        self.__check_number = check_number
        self.__bank_code = bank_code

    def get_check_number(self):
        return self.__check_number

class CashDeposit(Deposit):
    def __init__(self, cash_deposit_limit):
        self.__cash_deposit_limit = cash_deposit_limit

class Withdraw(Transaction):
    def __init__(self, amount):
        self.__amount = amount

    def get_amount(self):
        return self.__amount

class Transfer(Transaction):
    def __init__(self, destination_account_number):
        self.__destination_account_number = destination_account_number

    def get_destination_account(self):
        return self.__destination_account_number

```